LMS Backend
Entity Reference Guide

Complete database schema, relationships & API reference

Spring Boot · JPA · PostgreSQL

■ TABLE OF CONTENTS

#	Section	Assigned To
1	Department & Role	Ganesh
2	User Account & User Role	Chetan
3	Course & Module	Nideesh
4	Course Content & Assignment	Aniket
5	Enrolment & User Progress	Anuj
6	Test & Test Attempt	Shubham
7	Question & Student Answer	Vishank
8	Leaderboard & Assignment Submission	Aditya
9	Feedback & User Progress (detail)	Neeraj
10	Discussion Thread & Post	Yohaan

■■ 1. Department & Role

Assigned to: Ganesh

Department → department

An organisational unit within the institution. Courses and users are grouped under departments.

Fields

Field	Java Type	DB Column	Description
`departmentId`	`Integer`	`department_id` PK	Auto-generated unique identifier
`name`	`String`	`name` NOT NULL UNIQUE	Department name e.g. 'Computer Science'
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable

Business Rules

• name must be unique — enforced by @Column(unique=true)

• name is non-null — every department must have a name

• createdAt is auto-set via @PrePersist and must never be changed

• Deleting a department requires care — courses and users still reference it

API Endpoints

Method + Path	Description
`GET /api/departments`	List all departments
`POST /api/departments`	Create department (admin only)
`GET /api/departments/:id`	Get department by ID
`PUT /api/departments/:id`	Update department name
`DELETE /api/departments/:id`	Delete department (admin only)
`GET /api/departments/:id/courses`	Get all courses in a department

Note: Department and Role are lookup/reference entities — read far more than written. Consider caching role lookups in the security filter chain.

Role → role

Defines a named access level. Every user is assigned one or more roles via the UserRole join table.

Fields

Field	Java Type	DB Column	Description
`roleId`	`Integer`	`role_id` PK	Auto-generated unique identifier
`name`	`String`	`name` NOT NULL UNIQUE	Role identifier: ADMIN, INSTRUCTOR, STUDENT

Business Rules

• name must be unique — @Column(unique=true)

• Roles are seeded at startup via DataInitializer, not created by regular users

• Only Admins should assign or revoke roles via the UserRole table

API Endpoints

Method + Path	Description
`GET /api/roles`	List all roles (admin only)
`POST /api/roles`	Create a role (admin only)
`DELETE /api/roles/:id`	Delete a role (admin only)

Note: Seed initial roles (ADMIN, INSTRUCTOR, STUDENT) via a CommandLineRunner or Flyway migration. departmentId is Integer, not Long — be consistent in DTOs.

■ 2. User Account & User Role

Assigned to: Chetan

UserAccount → user_account

Core identity entity. Represents any person on the platform — student, instructor, or admin.

Fields

Field	Java Type	DB Column	Description
`userId`	`Long`	`user_id` PK	Auto-generated unique identifier
`fullName`	`String`	`full_name`	Full display name (single field, no first/last split)
`email`	`String`	`email` NOT NULL UNIQUE	Login identifier
`password`	`String`	`password` NOT NULL	Bcrypt hash — never plain text
`department`	`Department`	`dept_id` FK	Department the user belongs to
`isActive`	`Boolean`	`is_active`	Account enabled flag (default: true)
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable
`lastLogin`	`LocalDateTime`	`last_login`	Timestamp of most recent successful login

Business Rules

• email must be unique — enforced at DB level

• isActive defaults to true via @PrePersist; false = deactivated

• Deactivated accounts are rejected at login

• lastLogin must be updated on every successful login

• password must NEVER be included in API responses — use @JsonIgnore

API Endpoints

Method + Path	Description
`POST /api/auth/register`	Register a new user
`POST /api/auth/login`	Login, returns JWT
`POST /api/auth/refresh`	Refresh access token
`POST /api/auth/logout`	Invalidate refresh token
`GET /api/users/me`	Get own profile
`PUT /api/users/me`	Update own profile
`GET /api/users/:id`	Admin: get any user profile
`PATCH /api/users/:id/status`	Admin: activate/deactivate account

Note: JWT strategy — Access token: 15 min, contains userId/roles/dept_id. Refresh token: 7 days, store in HttpOnly cookie or DB table. Angular: store access token in memory, never localStorage.

UserRole → user_role

Join/pivot table mapping UserAccount to Role. Enables a user to hold multiple roles.

Fields

Field	Java Type	DB Column	Description
`userRoleId`	`Long`	`user_role_id` PK	Auto-generated unique identifier
`user`	`UserAccount`	`user_id` FK NOT NULL	The user being assigned the role
`role`	`Role`	`role_id` FK NOT NULL	The role being assigned
`assignedAt`	`LocalDateTime`	`assigned_at`	Auto-set on assignment, not updatable

Business Rules

- Unique constraint on (user_id, role_id) — prevents duplicate assignments
- assignedAt is auto-set and must never be modified
- Only Admins should assign or revoke roles

API Endpoints

Method + Path	Description
`GET /api/users/:id/roles`	List roles for a user
`POST /api/users/:id/roles`	Assign role (admin only)
`DELETE /api/users/:id/roles/:roleId`	Remove role (admin only)

Note: No assignedBy field exists in the current schema.

■ 3. Course & Module

Assigned to: Nideesh

Course → course

Top-level educational unit. A structured learning program that learners can enroll in.

Fields

Field	Java Type	DB Column	Description
`courseId`	`Long`	`course_id` PK	Auto-generated unique identifier
`title`	`String`	`title` NOT NULL	Name of the course
`description`	`String (TEXT)`	`description`	Detailed overview of the course
`imageUrl`	`String (TEXT)`	`image_url`	URL to the course cover image
`department`	`Department`	`department_id` FK	Department this course belongs to
`creator`	`UserAccount`	`creator_id` FK	User (instructor/admin) who created it
`isPublished`	`Boolean`	`is_published`	Whether students can see/enroll (default: false)
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable

Business Rules

• title is non-null — every course must have a name

• A course cannot be enrolled in unless isPublished = true

• Only Instructor/Admin roles should create or update a course

• Deleting a course should cascade-delete or archive its modules

API Endpoints

Method + Path	Description
`GET /api/courses`	List all published courses
`POST /api/courses`	Create a new course
`GET /api/courses/:id`	Get course by ID
`PUT /api/courses/:id`	Update course
`DELETE /api/courses/:id`	Delete/archive course
`GET /api/courses/:id/modules`	Get all modules in a course

Note: Consider soft-delete strategy to preserve enrollment history. imageUrl is TEXT — no character limit.

Module → module

Sub-unit of a Course. Groups related content into logical sections with a defined order.

Fields

Field	Java Type	DB Column	Description
`moduleId`	`Long`	`module_id` PK	Auto-generated unique identifier
`course`	`Course`	`course_id` FK NOT NULL	The course this module belongs to
`title`	`String`	`title`	Name of the module
`position`	`Integer`	`position`	Sequence/order within the course
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable

Business Rules

- Modules should be ordered by position when displayed
- Reordering updates position values across multiple records
- course_id is non-null — a module must always belong to a course
- Re-normalise position field on deletion to avoid gaps

API Endpoints

Method + Path	Description
`GET /api/modules/:id`	Get module by ID
`POST /api/courses/:id/modules`	Create module under a course
`PUT /api/modules/:id`	Update module
`DELETE /api/modules/:id`	Delete module
`GET /api/modules/:id/contents`	Get all content items in this module

Note: Use optimistic UI updates for drag-and-drop reordering. Position field should be re-normalised on deletion.

■ 4. Course Content & Assignment

Assigned to: Aniket

CourseContent → course_content

Actual learning material inside a Module. Can be a VIDEO, PDF, or QUIZ. Atomic unit of student progress.

Fields

Field	Java Type	DB Column	Description
`contentId`	`Long`	`content_id` PK	Auto-generated unique identifier
`module`	`Module`	`module_id` FK NOT NULL	The module this content belongs to
`contentType`	`String`	`content_type`	VIDEO, PDF, or QUIZ
`title`	`String`	`title`	Name/label of the content item
`filePath`	`String (TEXT)`	`file_path`	Storage path or URL to the file
`durationSeconds`	`Integer`	`duration_seconds`	Length in seconds (for videos)
`position`	`Integer`	`position`	Order within the module
`uploadedAt`	`LocalDateTime`	`uploaded_at`	Auto-set on creation, not updatable

Business Rules

• module_id is non-null — content must always belong to a module

• contentType must be VIDEO, PDF, or QUIZ

• position determines order within module

• durationSeconds is used by UserProgress to calculate video progress

• uploadedAt must never be set by client

API Endpoints

Method + Path	Description
`GET /api/contents/:id`	Get content item
`POST /api/modules/:id/contents`	Add content to a module
`PUT /api/contents/:id`	Update content metadata
`DELETE /api/contents/:id`	Delete content item

Note: filePath is TEXT — use cloud storage keys (S3) not filesystem paths. durationSeconds enables progress bar and resume.

Assignment → assignment

Task given to students as part of a Module. Students submit work by a deadline for instructor grading.

Fields

Field	Java Type	DB Column	Description
`assignmentId`	`Long`	`assignment_id` PK	Auto-generated unique identifier
`module`	`Module`	`module_id` FK NOT NULL	The module this assignment belongs to
`title`	`String`	`title` NOT NULL	Assignment name
`description`	`String (TEXT)`	`description`	Detailed instructions
`maxMarks`	`Integer`	`max_marks`	Maximum marks that can be awarded
`dueDate`	`LocalDateTime`	`due_date`	Submission deadline
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable

Business Rules

• title and module_id are non-null

• dueDate should be validated as a future date on creation

• maxMarks must be greater than 0

• Each student may have only ONE submission per assignment

• Assignment belongs to Module, not directly to Course

API Endpoints

Method + Path	Description
`GET /api/assignments/:id`	Get assignment details
`POST /api/modules/:id/assignments`	Create assignment under a module
`PUT /api/assignments/:id`	Update assignment
`DELETE /api/assignments/:id`	Delete assignment
`GET /api/assignments/:id/submissions`	Get all submissions

Note: dueDate should be stored and compared as UTC. Decide cascade-delete policy when a module is deleted.

■ 5. Enrolment & User Progress

Assigned to: Anuj

Enrollment → enrollment

Records the relationship between a User and a Course, and the role they have in that course.

Fields

Field	Java Type	DB Column	Description
`enrollmentId`	`Long`	`enrollment_id` PK	Auto-generated unique identifier
`user`	`UserAccount`	`user_id` FK NOT NULL	The user who enrolled
`course`	`Course`	`course_id` FK NOT NULL	The course they enrolled in
`role`	`String`	`role` NOT NULL	STUDENT or INSTRUCTOR in this course
`enrolledAt`	`LocalDateTime`	`enrolled_at`	Auto-set via @PrePersist, not updatable

Business Rules

• Unique constraint on (user_id, course_id, role) — no duplicate enrollments in same role

• role must be STUDENT or INSTRUCTOR

• enrolledAt is auto-set and must never be modified

• STUDENT can take tests/submit assignments; INSTRUCTOR can create them

API Endpoints

Method + Path	Description
`POST /api/enrollments`	Enroll in a course
`GET /api/enrollments/me`	Get my enrollments
`DELETE /api/enrollments/:id`	Unenroll from a course
`GET /api/courses/:id/enrollments`	Admin: all enrollments for a course

Note: No status, completionPercentage, or certificateUrl fields exist in the current entity.

UserProgress → user_progress

Tracks a student's progress on a specific CourseContent item — seconds watched, percent complete.

Fields

Field	Java Type	DB Column	Description
`progressId`	`Long`	`progress_id` PK	Auto-generated unique identifier
`user`	`UserAccount`	`user_id` FK NOT NULL	The student whose progress this is
`content`	`CourseContent`	`content_id` FK NOT NULL	Content item being tracked
`completedAt`	`LocalDateTime`	`completed_at`	When content was fully completed (nullable)
`progressPercent`	`BigDecimal(5,2)`	`progress_percent`	Percentage consumed — 0.00 to 100.00
`secondsWatched`	`Integer`	`seconds_watched`	Total seconds of video watched

Business Rules

- Unique constraint on (user_id, content_id) — one record per student per content
- If record exists, UPDATE it (upsert) — do not insert again
- completedAt is null until student finishes — set when progressPercent >= 90%
- secondsWatched supports resume functionality for videos

API Endpoints

Method + Path	Description
`POST /api/progress`	Upsert progress for content item
`GET /api/progress/me`	Get all my progress records
`GET /api/progress/content/:id`	My progress on specific content

Note: BigDecimal(5,2) — max value 999.99; clamp input to 100.00. Aggregate across all content to derive overall course completion percentage (not stored — computed on the fly).

■ 6. Test & Test Attempt

Assigned to: Shubham

Test → test

Timed assessment within a Course. Evaluates student understanding via a set of Questions.

Fields

Field	Java Type	DB Column	Description
`testId`	`Long`	`test_id` PK	Auto-generated unique identifier
`course`	`Course`	`course_id` FK NOT NULL	The course this test belongs to
`title`	`String`	`title`	Name of the test
`durationMinutes`	`Integer`	`duration_minutes`	Time limit in minutes
`securityPolicy`	`String`	`security_policy`	Security settings (ExamGuard integration)
`scheduledAt`	`LocalDateTime`	`scheduled_at`	When test opens (null = immediately available)
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable

Business Rules

• course_id is non-null — test must always belong to a course

• If scheduledAt is set, test is not accessible before that datetime

• durationMinutes enforced server-side — never trust the client

• No totalMarks, passingMarks, maxAttempts, or isPublished fields exist

API Endpoints

Method + Path	Description
`GET /api/tests/:id`	Get test details
`POST /api/courses/:id/tests`	Create test under a course
`PUT /api/tests/:id`	Update test
`DELETE /api/tests/:id`	Delete test
`GET /api/tests/:id/questions`	Get all questions in a test
`GET /api/courses/:id/tests`	List all tests in a course

Note: securityPolicy is used by the ExamGuard VS Code extension. Consider indexing (user_id, test_id) for fast attempt lookups.

TestAttempt → test_attempt

One sitting of a student taking a Test. Tracks start/end time, score, and status.

Fields

Field	Java Type	DB Column	Description
`attemptId`	`Long`	`attempt_id` PK	Auto-generated unique identifier
`user`	`UserAccount`	`user_id` FK NOT NULL	The student attempting the test
`test`	`Test`	`test_id` FK NOT NULL	The test being attempted
`startedAt`	`LocalDateTime`	`started_at`	When the student started
`completedAt`	`LocalDateTime`	`completed_at`	When finished (nullable until done)
`score`	`BigDecimal(8,2)`	`score`	Marks achieved (server-calculated)
`status`	`String`	`status`	IN_PROGRESS, COMPLETED, or ABANDONED

Business Rules

- Status transitions: IN_PROGRESS → COMPLETED (submit) or IN_PROGRESS → ABANDONED (timeout)
- completedAt is null while IN_PROGRESS; set on completion or abandonment
- score is calculated server-side from StudentAnswer.marksAwarded — never accept client score
- Backend should auto-transition to ABANDONED if startedAt + durationMinutes has elapsed

API Endpoints

Method + Path	Description
`POST /api/tests/:id/attempts`	Start a new attempt
`GET /api/attempts/:id`	Get attempt details + answers
`POST /api/attempts/:id/submit`	Submit/complete the attempt
`GET /api/tests/:id/attempts/me`	Get my attempts for a test
`GET /api/tests/:id/attempts`	Admin: all attempts for a test

Note: Use server-side timer: record startedAt, check now <= startedAt + durationMinutes on submission. score uses BigDecimal(8,2).

■ 7. Question & Student Answer

Assigned to: Vishank

Question → question

Single item within a Test. Presents a problem and expects a student response. Type determines grading.

Fields

Field	Java Type	DB Column	Description
`questionId`	`Long`	`question_id` PK	Auto-generated unique identifier
`test`	`Test`	`test_id` FK NOT NULL	The test this question belongs to
`body`	`String (TEXT)`	`body`	The question text/prompt
`questionType`	`String`	`question_type`	MCQ, TEXT, or TF (True/False)
`marks`	`Integer`	`marks`	Points for correct answer (must be > 0)
`correctAnswer`	`String (TEXT)`	`correct_answer`	Correct answer — NEVER sent to client pre-submission
`extra`	`String (JSON)`	`extra`	JSON field: MCQ options array, hints, etc.

Business Rules

• test_id is non-null — question must always belong to a test

• marks must be > 0

• For MCQ: extra must contain valid JSON array; correctAnswer must match one option

• For TF: correctAnswer must be 'true' or 'false'

• correctAnswer must NEVER be sent to client before test submission

• No separate QuestionOption table — MCQ options stored in extra JSON field

API Endpoints

Method + Path	Description
`GET /api/questions/:id`	Get question (correctAnswer hidden for students)
`POST /api/tests/:id/questions`	Add question to a test
`PUT /api/questions/:id`	Update question
`DELETE /api/questions/:id`	Delete question
`GET /api/tests/:id/questions`	Get all questions in a test

Note: extra is stored as a json column — parse it in the service layer. For MCQ grading, do case-insensitive/trimmed string comparison.

StudentAnswer → student_answer

Records the response a student gave to a Question during a TestAttempt. Stores answer and marks awarded.

Fields

Field	Java Type	DB Column	Description
`studentAnswerId`	`Long`	`student_answer_id` PK	Auto-generated unique identifier
`attempt`	`TestAttempt`	`attempt_id` FK NOT NULL	The test attempt this answer belongs to
`question`	`Question`	`question_id` FK NOT NULL	The question being answered
`answerText`	`String (TEXT)`	`answer_text`	Student's written or selected answer
`marksAwarded`	`BigDecimal(8,2)`	`marks_awarded`	Marks given (null until graded)

Business Rules

- attempt_id and question_id are non-null
- marksAwarded is null until graded; BigDecimal(8,2) for precision
- MCQ/TF: auto-graded server-side on submission
- TEXT: marksAwarded stays null until instructor manually grades
- Sum of all marksAwarded = TestAttempt.score
- No selectedOptionId, isCorrect, or answeredAt field exists

API Endpoints

Method + Path	Description
`POST /api/attempts/:id/answers`	Submit answers for an attempt
`GET /api/attempts/:id/answers`	Get all answers (instructor/admin)
`PATCH /api/answers/:id/grade`	Instructor: manually set marksAwarded

Note: Recalculate TestAttempt.score after all auto-gradable answers are processed. All answer types (including MCQ) stored as text in answerText.

■ 8. Leaderboard & Assignment Submission

Assigned to: Aditya

Leaderboard → leaderboard

Tracks a student's performance ranking within a Course — average test score and global/dept rank.

Fields

Field	Java Type	DB Column	Description
`leaderboardId`	`Long`	`leaderboard_id` PK	Auto-generated unique identifier
`user`	`UserAccount`	`user_id` FK NOT NULL	The student this entry belongs to
`course`	`Course`	`course_id` FK NOT NULL	The course this ranking is for
`avgScore`	`BigDecimal(6,2)`	`avg_score`	Average score across all completed test attempts
`globalRank`	`Integer`	`global_rank`	Rank among ALL students in this course
`deptRank`	`Integer`	`dept_rank`	Rank among students in same department
`updatedAt`	`LocalDateTime`	`updated_at`	When this entry was last recalculated

Business Rules

• One leaderboard record per (user, course) — unique constraint required

• avgScore = average of TestAttempt.score for COMPLETED attempts in this course

• globalRank = position when all enrolled students sorted by avgScore DESC

• deptRank = same but scoped to students from same department

• Ranks are computed/recalculated — never set manually

• updatedAt refreshed every time a new TestAttempt is completed

API Endpoints

Method + Path	Description
`GET /api/courses/:id/leaderboard`	Get ranked leaderboard for a course
`GET /api/leaderboard/me`	Get my leaderboard entries across all courses

Note: Recalculate ranks asynchronously (scheduled job or event after TestAttempt completion) — not on every API call. avgScore uses BigDecimal(6,2).

AssignmentSubmission → assignment_submission

Student's response to an Assignment. Holds submitted content, marks awarded, and instructor feedback.

Fields

Field	Java Type	DB Column	Description
`submissionId`	`Long`	`submission_id` PK	Auto-generated unique identifier
`assignment`	`Assignment`	`assignment_id` FK NOT NULL	The assignment being submitted
`student`	`UserAccount`	`student_id` FK NOT NULL	The student who submitted
`submissionText`	`String (TEXT)`	`submission_text`	Written/text-based answer content
`fileUrl`	`String (TEXT)`	`file_url`	URL to uploaded file (PDF, zip, etc.)
`marksAwarded`	`Integer`	`marks_awarded`	Marks given by instructor (null until graded)
`feedback`	`String (TEXT)`	`feedback`	Instructor's written feedback
`submittedAt`	`LocalDateTime`	`submitted_at`	Auto-set on creation, never changeable

Business Rules

- assignment_id and student_id are non-null
- One submission per assignment per student — unique constraint on (student_id, assignment_id)
- At least one of submissionText or fileUrl must be non-null
- marksAwarded is null until instructor grades
- submittedAt is auto-set and must never be changed

API Endpoints

Method + Path	Description
`POST /api/assignments/:id/submissions`	Submit an assignment
`GET /api/submissions/:id`	Get submission details
`GET /api/assignments/:id/submissions`	Instructor: list all submissions
`PATCH /api/submissions/:id/grade`	Instructor: set marksAwarded + feedback
`GET /api/submissions/me`	Student: get all my submissions

Note: Use pre-signed cloud storage URLs (S3/GCS) for secure file upload and download. Consider adding gradedAt timestamp.

■ 9. Feedback & User Progress (Detail)

Assigned to: Neeraj

Feedback → feedback

Rating and comment a student leaves on a Course. Helps instructors understand course reception.

Fields

Field	Java Type	DB Column	Description
`feedbackId`	`Long`	`feedback_id` PK	Auto-generated unique identifier
`user`	`UserAccount`	`user_id` FK NOT NULL	The student who left the feedback
`course`	`Course`	`course_id` FK NOT NULL	The course being reviewed
`rating`	`Short`	`rating`	Numeric rating 1–5
`message`	`String (TEXT)`	`message`	Optional written comment (nullable)
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable

Business Rules

• user_id and course_id are non-null

• rating must be in range 1–5 (inclusive) — enforce in service layer

• message is optional (nullable)

• Typically one feedback per course per user — unique constraint on (user_id, course_id)

• Average rating: AVG(rating) WHERE course_id = ?

API Endpoints

Method + Path	Description
`POST /api/courses/:id/feedback`	Submit feedback for a course
`GET /api/courses/:id/feedback`	Get all feedback for a course
`GET /api/feedback/:id`	Get specific feedback entry
`DELETE /api/feedback/:id`	Delete feedback (admin/author only)
`GET /api/courses/:id/feedback/avg`	Get average rating for a course

Note: Average rating is best computed in a repository query (@Query) rather than loading all records into memory.

UserProgress → user_progress

How far a student is within a specific CourseContent item. Enables resume and completion tracking.

Fields

Field	Java Type	DB Column	Description
`progressId`	`Long`	`progress_id` PK	Auto-generated unique identifier
`user`	`UserAccount`	`user_id` FK NOT NULL	The student whose progress this is
`content`	`CourseContent`	`content_id` FK NOT NULL	Content item being tracked
`completedAt`	`LocalDateTime`	`completed_at`	Timestamp when marked completed (nullable)
`progressPercent`	`BigDecimal(5,2)`	`progress_percent`	Percentage consumed — 0.00 to 100.00
`secondsWatched`	`Integer`	`seconds_watched`	Total seconds watched (video content)

Business Rules

• Unique constraint on (user_id, content_id) — one record per user per content item

• Upsert pattern — if record exists, UPDATE; else INSERT

• completedAt set when progressPercent = 100

• secondsWatched used for video resume functionality

API Endpoints

Method + Path	Description
`POST /api/progress`	Upsert progress for content item
`GET /api/progress/me`	Get all my progress records
`GET /api/progress/content/:id`	My progress on specific content
`GET /api/progress/course/:id/summary`	Admin: progress summary across all students

Note: Use INSERT ... ON CONFLICT DO UPDATE or JPA merge for upserts. Consider adding updatedAt to track when progress was last recorded.

■ 10. Discussion Thread & Post

Assigned to: Yohaan

DiscussionThread → discussion_thread

Top-level conversation topic attached to a Course. Container for Posts (replies). Like a forum thread.

Fields

Field	Java Type	DB Column	Description
`threadId`	`Long`	`thread_id` PK	Auto-generated unique identifier
`course`	`Course`	`course_id` FK	The course this thread belongs to
`user`	`UserAccount`	`user_id` FK	The user who created the thread
`title`	`String`	`title`	Subject/title of the thread
`content`	`String (TEXT)`	`content`	Body/opening message of the thread
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable
`updatedAt`	`LocalDateTime`	`updated_at`	Last modified timestamp

Business Rules

• title and content are both required (non-null)

• createdAt is set automatically — never set by client

• updatedAt should be updated when title or content is edited

• Only thread creator or admin/moderator can edit or delete

• Deleting a thread should cascade-delete all its Posts

API Endpoints

Method + Path	Description
`GET /api/threads/:id`	Get thread + posts
`POST /api/courses/:id/threads`	Create thread in a course
`PUT /api/threads/:id`	Update thread (creator/admin only)
`DELETE /api/threads/:id`	Delete thread + its posts
`GET /api/courses/:id/threads`	List all threads in a course

Note: Implement soft-delete to preserve thread structure when a parent post is removed.

Post → post

Reply within a DiscussionThread. Supports nested replies via parentPost (self-referential).

Fields

Field	Java Type	DB Column	Description
`postId`	`Long`	`post_id` PK	Auto-generated unique identifier
`thread`	`DiscussionThread`	`thread_id` FK NOT NULL	The thread this post belongs to
`user`	`UserAccount`	`user_id` FK NOT NULL	The user who wrote the post
`body`	`String (TEXT)`	`body`	The text content of the post
`parentPost`	`Post`	`parent_post_id` FK	Parent post for nested replies (null = top-level)
`createdAt`	`LocalDateTime`	`created_at`	Auto-set on creation, not updatable

Business Rules

• thread_id and user_id are non-null

• parentPost is nullable — null = direct reply to thread; non-null = nested reply

• body must not be empty or blank

• Only post author or admin can edit/delete a post

• parentPost creates a self-referential @ManyToOne relationship

API Endpoints

Method + Path	Description
`POST /api/threads/:id/posts`	Create post in a thread
`PUT /api/posts/:id`	Edit post (author/admin only)
`DELETE /api/posts/:id`	Delete post
`GET /api/threads/:id/posts`	Get all posts in a thread

Note: Sort posts by createdAt ASC and nest replies under parentPost. Soft-delete recommended to preserve thread structure.